Chapter Six
Test Management

What is Test Management?

- The process of organizing, controlling, and tracking the testing activities of a project.
- The 4 Key Pillars:
  - Planning (Strategy & Scope)
  - Resource Management (People & Environment)
  - Execution (Running tests)
  - Reporting (Metrics & Status)

- Not just “running tests” – it includes:
  - Defining strategy and scope
  - Estimating effort and cost
  - Assigning people and environments
  - Tracking progress against plan
  - Communicating status to stakeholders
- Without test management, testing becomes chaotic, late, and unreliable.

Give a one-sentence example of what 'chaotic testing' looks like in practice.

Pillar	Key Activities	Typical Deliverable
Planning	Risk analysis, strategy selection, estimation, scheduling	Test Plan document
Resource Management	Hiring, training, assignment, environment provisioning	Resource allocation matrix
Execution Control	Test case execution, defect logging, progress monitoring	Daily status log
Reporting	Metrics collection, dashboard updates, sign-off recommendations	Test Summary Report

Test Management vs. Project Management

- **Project Manager cares about**: budget, timeline, scope, resources across all disciplines (dev, QA…).
- **Test Manager cares about:** test coverage, defect severity, environment uptime, and test team productivity.
- **Overlap:** Both need to know about critical blockers and release risks.
- **Testing** is just a line in the project plan. Testing becomes invisible until the end.

Resource Management

- What is a "Test Resource"?
  - Human: Testers, Automation Engineers, SMEs (Subject Matter Experts).
  - Environmental: Test Servers, Staging DBs.
  - Tools: License keys, CI/CD pipelines.
  - Data: Test data sets, masking rules.

Resource Name	Role	Mon	Tue	Wed	Thu	Fri	Total Hours	Notes
x	Senior Manual Tester	8h	8h	8h	8h	4h	36h	Half day Friday (training)
y	Senior Manual Tester	8h	8h	8h	0h	0h	24h	Sick Tue–Fri (planned)
z	Automation Engineer	8h	8h	8h	8h	8h	40h	Overloaded – see risks
a	Junior Manual Tester	8h	8h	8h	8h	0h	32h	Needs supervision
b	Performance Specialist	0h	8h	8h	8h	0h	24h	On loan from DevOps only Wed–Fri
c	Test Data Manager	4h	4h	4h	4h	4h	20h	Part-time on this project

Resource Type	Item	Assigned To (if shared)	Availability	Notes
Environment	Staging server (Linux, 16GB RAM)	Test team + Dev team	8 AM – 6 PM weekdays	Shared – no exclusive access
Environment	Performance lab (8 VMs)	Performance specialist only	By reservation only	2 slots per day max
Tool seat	Jira + Xray license	10 users total (all testers)	100%	No additional licenses available
Tool seat	LoadRunner license	1 concurrent user	8 AM – 5 PM	Only Morgan can use it
Data	Synthetic patient data set	Entire team	Always available	10,000 records
Data	Anonymized production dump	Test data manager only	Once per week	Requires 2 hours to refresh

The 3 Challenges of Resource Management

- **The Bottleneck**:
  - 3 testers vs. 50 new bugs (Over-allocation)
  - A development team just finished a large feature merge. The bug tracker shows 50 new issues logged overnight. Your test team has only 3 people. Each bug takes, on average, 30 minutes to reproduce, document, and verify a fix later. That's 25 hours of work – but there are only 24 hours in a day, and testers also have meetings, regression tests, and exploratory testing.

- Why it happens?
  - Uneven flow – Developers deliver work in big batches instead of small increments.
  - No bug triage – Every bug, regardless of severity, is treated as “must test now.”
  - Understaffing – The team was sized for average load, not peak load.
  - Manual processes – No automation to smoke-test or filter duplicates.

- **The Skills Gap:**
  - **Eg**. Need automation expert, but only manual testers available.

- **Environment Availability:**
  - Eg. "The staging server is down again!“

Test Reporting and Tools

- Test reporting is the process of summarizing and communicating the **activities, results, and status** of testing efforts to project stakeholders.
- It transforms raw test execution data into meaningful information that supports decision-making
- A **test report** is a formal document that presents:
  - What was tested
  - How it was tested
  - What the results were
  - What defects remain
  - Whether the software is ready for release

Purpose & Importance of Test Reporting

Purpose	Explanation
Provide visibility	Give stakeholders a clear picture of testing progress and product quality
Support go/no-go decisions	Help management decide if a release can proceed
Identify risks	Highlight unresolved defects or coverage gaps
Improve accountability	Show what the test team has achieved
Guide future testing	Show where more testing or better strategies are needed

Key Components of a Test Report

- A professional test report typically includes:
  - **Header** – Project name, report version, date, author
  - **Executive Summary** – High-level status for management
  - **Test Scope** – What was tested and what was excluded
  - **Test Environment & Build** – Hardware/software configuration

- **Test Execution Summary** – Pass/fail/blocked counts, run rate
- **Defect Summary** – Open, closed, critical, trend
- **Coverage Metrics** – Requirements coverage, code coverage, risk coverage
- **Known Risks & Issues** – Anything that might impact the release decision
- **Recommendations** – e.g., “Fix critical defects before release” or “Proceed with monitoring”
- **Appendices** – Logs, test case lists, traceability matrices

Common Test Metrics Used in Reports

- Metrics are the quantitative backbone of test reporting. Use them to show trends and health:

Metric	What it tells
Pass/Fail Rate	Percentage of test cases that passed vs. failed
Defect Density	Number of defects per module or per KLOC
Defect Leakage	Defects found in production after release – measures test effectiveness
Test Execution Progress	Number of tests planned vs. executed over time
Test Coverage	Percentage of requirements, code lines, or risks covered
Mean Time to Detect (MTTD)	Average time to find a defect
Mean Time to Repair (MTTR)	Average fix time – helps estimate closure

- **Visuals help** – use bar charts for pass/fail, line charts for defect trends, pie charts for status breakdown.

Types of Test Reports (by Audience & Timing)

Report Type	Audience	Frequency	Focus
Daily Status Report	Test lead, project manager	Daily	Progress, blockers, executed vs planned
Weekly / Sprint Report	Project team, stakeholders	Weekly	Trends, defect ageing, risks
Master Test Report	Quality assurance, management	At major milestones (e.g., after each test cycle)	Full assessment of testing phase
Regression Test Report	Developers, test lead	After each regression run	Impact of changes, unexpected failures
Release / Sign-off Report	Steering committee, product owner	End of testing phase	Go/no-go recommendation

Best Practices for Effective Test Reporting

- **Know your audience**
  - Executives want a one-page summary with red/yellow/green status.
  - Technical leads want defect stack traces and environment details.
- **Be honest** – Never hide failures or bypassed tests. Incomplete testing is a risk – communicate it openly.
- **Automate data collection** – Use tools to pull metrics directly instead of manual spreadsheets. This saves time and prevents errors.
- **Focus on meaningful metrics** – Avoid “vanity metrics” (e.g., number of test cases written). Show coverage, defects, and business risk.

- **Use visualisation** – A line chart of defects found vs. fixed is more powerful than a table of numbers.
- **Compare to baseline** – Always show current results against previous periods (e.g., last sprint, last release) to highlight trends.
- **Include actionable recommendations** – Never end with just data. Say “Because we see X, we recommend Y.”

Common Pitfalls to Avoid

Pitfall	Consequence
Overloading with data	Stakeholders ignore the report
Inconsistent definitions	“Passed” means different things to different people
No trend analysis	Hard to see if quality is improving or degrading
Manual report creation	Takes too long, becomes outdated
Ignoring negative results	Creates false confidence, leads to production failures

Sample Report Structure (One-Page Executive View)

Project: TripsApp Mobile
Report Date: 2025-03-15
Build: 2.1.3
Status: Yellow – at risk

Metric	Value	Trend
Planned test cases	320	–
Executed	305 (95%)	↗
Passed	240 (79%)	↘
Failed	40 (13%)	↗
Blocked	25 (8%)	↘
Open critical defects	4	↗
Defects found this week	12	↘
Defects fixed this week	8	–
Requirements coverage	92%	–

Key Risks:

- Payment module has 3 open critical defects – no workaround.
- Environment unstable after 03-14 patch.

Recommendation:

- Do not release until payment module defects are fixed and regression passes.